2-D array operations program using a Jagged Array

```
package com.company.array;

import java.util.Scanner;

public class JaggedArrayOperations {

    // Display the jagged array
    public static void display(int[][] arr) {

        System.out.println("\nJagged Array Elements:");

        for (int i = 0; i < arr.length; i++) {

            System.out.print("Row " + i + ": ");

            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + "\t");
            }

            System.out.println();
        }
    }

    // Calculate the sum of all elements
    public static int calculateSum(int[][] arr) {

        int sum = 0;

        for (int i = 0; i < arr.length; i++) {

            for (int j = 0; j < arr[i].length; j++) {
                sum += arr[i][j];
            }
        }

        return sum;
    }

    // Calculate sum of each row
    public static void rowWiseSum(int[][] arr) {

        System.out.println("\nRow-wise Sum:");

        for (int i = 0; i < arr.length; i++) {

            int sum = 0;

            for (int j = 0; j < arr[i].length; j++) {
                sum += arr[i][j];
            }

            System.out.println("Sum of Row " + i + " = " + sum);
        }
    }
```

```
// Find maximum element
public static int findMaximum(int[][] arr) {

    int max = arr[0][0];

    for (int i = 0; i < arr.length; i++) {

        for (int j = 0; j < arr[i].length; j++) {

            if (arr[i][j] > max) {
                max = arr[i][j];
            }
        }
    }

    return max;
}

// Find minimum element
public static int findMinimum(int[][] arr) {

    int min = arr[0][0];

    for (int i = 0; i < arr.length; i++) {

        for (int j = 0; j < arr[i].length; j++) {

            if (arr[i][j] < min) {
                min = arr[i][j];
            }
        }
    }

    return min;
}

// Search an element
public static void searchElement(int[][] arr, int key) {

    boolean found = false;

    for (int i = 0; i < arr.length; i++) {

        for (int j = 0; j < arr[i].length; j++) {

            if (arr[i][j] == key) {

                System.out.println(
                        "Element " + key +
                        " found at [" + i + "][" + j + "]"
                );

                found = true;
            }
        }
    }
```

```
    if (!found) {
        System.out.println("Element " + key + " not found.");
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    /*
     * Creating a jagged array.
     *
     * The array has 4 rows.
     * Each row can have a different number of elements.
     *
     * Row 0 → 4 elements
     * Row 1 → 3 elements
     * Row 2 → 2 elements
     * Row 3 → 1 element
     */
    int[][] arr = new int[4][];

    arr[0] = new int[4];
    arr[1] = new int[3];
    arr[2] = new int[2];
    arr[3] = new int[1];

    /*
     * Taking input for each row.
     *
     * Notice that arr[i].length is used
     * instead of a fixed column size.
     */
    System.out.println("Enter elements:");

    for (int i = 0; i < arr.length; i++) {

        System.out.println(
                "Enter " + arr[i].length +
                " elements for Row " + i + ":"
        );

        for (int j = 0; j < arr[i].length; j++) {

            System.out.print(
                    "Element [" + i + "][" + j + "]: "
            );

            arr[i][j] = sc.nextInt();
        }
    }

    // Display the jagged array
    display(arr);

    // Calculate total sum
```

```
        System.out.println(
                "\nTotal Sum = " + calculateSum(arr)
        );

        // Calculate row-wise sum
        rowWiseSum(arr);

        // Find maximum
        System.out.println(
                "\nMaximum Element = " + findMaximum(arr)
        );

        // Find minimum
        System.out.println(
                "Minimum Element = " + findMinimum(arr)
        );

        // Search an element
        System.out.print("\nEnter element to search: ");
        int key = sc.nextInt();

        searchElement(arr, key);

        sc.close();
    }
}
```

Example

Suppose the user enters:

```
10 20 30 40
50 60 70
80 90
100
```

The jagged array is:

```
Row 0 → 10  20  30  40
Row 1 → 50  60  70
Row 2 → 80  90
Row 3 → 100
```

Important Difference from a Normal 2-D Array

A rectangular 2-D array would be:

```
int[][] arr = new int[4][4];
```

Every row has **4 elements**.

In a jagged array:

```
int[][] arr = new int[4][];

arr[0] = new int[4];
arr[1] = new int[3];
arr[2] = new int[2];
arr[3] = new int[1];
```

The rows have different sizes:

```
        0   1   2   3

Row 0   10  20  30  40
Row 1   50  60  70
Row 2   80  90
Row 3   100
```

□ Key Teaching Point

When working with a jagged array, **never assume that every row has the same number of columns**.

Use:

```
for (int i = 0; i < arr.length; i++) {

    for (int j = 0; j < arr[i].length; j++) {

        // process arr[i][j]
    }
}
```

Here:

- `arr.length` → number of **rows**
- `arr[i].length` → number of **elements in the current row**

This is the most important concept students should understand when moving from a rectangular 2-D array to a jagged array.